

Supporting Information: ripopt — An Interior Point Optimizer in Rust with Implicit Slacks and Robust Recovery

John Kitchen

May 22, 2026

Contents

1	Introduction	4
2	Implicit Slack Formulation	6
2.1	Algorithm Description	6
2.2	Source Code References	6
2.3	Condensed KKT via Schur Complement (legacy)	6
3	Convergence Test	7
3.1	Algorithm Description	7
3.1.1	Dual Scaling s_d	7
3.1.2	Two-Gate Test	7
3.2	Source Code References	7
4	Two-Phase Restoration with Multi-Attempt Recovery	8
4.1	Algorithm Description	8
4.1.1	Phase 1: Gauss-Newton Feasibility Solver	8
4.1.2	Phase 2: NLP Restoration	8
4.1.3	Multi-Attempt Recovery	8
4.1.4	Post-Restoration State Reset	9
4.2	Source Code References	9
5	Multi-Solver Fallback Architecture	9
5.1	Algorithm Description	9
5.1.1	Best-Point Tracking	9
5.1.2	Dual Stagnation Detection	10
5.2	Source Code References	10

6	Adaptive Barrier Parameter	10
6.1	Algorithm Description	10
6.1.1	Barrier Oracle with Mehrotra Integration	10
6.1.2	Mehrotra Predictor-Corrector and Gondzio Corrections	10
6.1.3	Free and Fixed Modes	11
6.1.4	Fraction-to-Boundary	11
6.1.5	No-Bounds Special Case	11
6.2	Source Code References	11
7	Filter Line Search	11
7.1	Algorithm Description	11
7.1.1	Switching Condition	11
7.1.2	Second-Order Correction (SOC)	11
7.1.3	Objective Safeguard	12
7.2	Source Code References	12
8	Linear Algebra	12
8.1	Algorithm Description	12
8.1.1	Sparse Solver Dispatch	12
8.1.2	feral: pure-Rust Multifrontal LDL ^T (v0.8 default)	12
8.1.3	rmumps: Legacy Multifrontal LDL ^T (<code>feature"rmumps"=</code>)	13
8.1.4	Bunch-Kaufman LDL ^T Factorization (Dense)	13
8.1.5	Inertia Correction	13
8.1.6	Iterative Refinement	13
8.2	Source Code References	13
8.2.1	feral Integration (v0.8 default)	13
8.2.2	rmumps Integration (legacy, <code>feature"rmumps"=</code>)	14
8.2.3	Trait surface	14
8.2.4	Dense Solver	14
8.2.5	Inertia Correction and Search Direction	14
9	Implementation	14
9.1	Problem Interface	14
9.2	Public API	15
9.3	AMPL NL Interface and Pyomo Integration	15
9.4	Julia/JuMP Interface (Ripopt.jl)	16
10	Parametric Sensitivity Analysis	16
10.1	Algorithm Description	16
10.2	Source Code References	17
11	Running the Benchmarks	17
11.1	Prerequisites	17
11.2	Hock-Schittkowski (HS) Suite	17
11.2.1	Generating HS Problem Code	17

11.2.2	Running ripopt on HS Suite	17
11.2.3	Running Ipopt (native) on HS Suite	18
11.2.4	Running cyipopt on HS Suite	18
11.2.5	Comparing Results	18
11.3	CUTEst Benchmark Suite	18
11.3.1	Prerequisites	18
11.3.2	Running CUTEst Benchmarks	18
11.3.3	Interpreting Results	19
11.4	Unit Tests	19
11.5	Pyomo Integration	19
11.6	Julia/JuMP Integration	20

Contents

1 Introduction

This Supporting Information provides algorithmic details and source code references for each section of the manuscript. Source code locations are given as org-mode links in the format `[[file:line]]` that point to the ripopt repository root at `src/`. All line numbers refer to ripopt v0.8.2.

The ripopt source code is organized as follows:

File	Purpose
src/lib.rs	Public API (<code>solve</code> , <code>solve_with_sensitivity</code>)
src/problem.rs	<code>NlpProblem</code> trait definition
src/options.rs	<code>SolverOptions</code> struct
src/result.rs	<code>SolveResult</code> and <code>SolveStatus</code> types
src/ipm.rs	Core IPM engine, fallback dispatch, restoration cascade
src/kkt.rs	KKT assembly, inertia correction, dense condensed-KKT
src/kkt_aug.rs	Augmented-system assembly utilities
src/convergence.rs	Two-gate convergence test
src/filter.rs	Filter line search and fraction-to-boundary
src/restoration_nlp.rs	NLP restoration subproblem (Phase 2)
src/slack_formulation.rs	Explicit slack reformulation fallback
src/l1_penalty_barrier_nlp.rs	ℓ_1 penalty-barrier NLP (soft-restoration sub-NLP)
src/lbfgs.rs	L-BFGS solver
src/bc_solver.rs	Bound-constrained / augmented-Lagrangian fallback driver
src/preprocessing.rs	Fixed-variable elimination, bound tightening, redundancy reduction
src/auxiliary_preprocessing.rs	NL-side preprocessing (linear/quadratic extraction, scaling)
src/split_nlp.rs	Inequality-splitting reformulation
src/warmstart.rs	Warm-starting utilities
src/sensitivity.rs	Parametric sensitivity analysis (sIPOPT-style)
src/linear_solver/mod.rs	<code>KktMatrix</code> , <code>LinearSolver</code> trait, matvec
src/linear_solver/dense.rs	Dense Bunch-Kaufman LDL^T factorization
src/linear_solver/feral_direct.rs	feral multifrontal sparse LDL^T (v0.8 default)
src/linear_solver/feral_iterative.rs	Krylov iterative refinement around feral
src/linear_solver/feral_hybrid.rs	Hybrid direct/iterative driver around feral
src/linear_solver/multifrontal.rs	rmumps multifrontal sparse LDL^T (legacy, <code>feature="rmumps"</code>)
src/linear_solver/iterative.rs	Krylov iterative refinement around rmumps (legacy)
src/linear_solver/hybrid.rs	Hybrid direct/iterative driver around rmumps (legacy)
src/linear_solver/sparse.rs	faer sparse LDL^T (optional dense/sparse primitives)
src/linear_solver/banded.rs	Banded LDL^T factorization
src/c_api.rs	C API bindings for external language integration
src/nl/parser.rs	AMPL NL file parser
src/nl/problem_impl.rs	<code>NlProblem</code> (<code>NlpProblem</code> for NL files)
src/nl/autodiff.rs	Automatic differentiation for NL expressions
src/bin/riopt_ampl.rs	AMPL binary interface
gams/gams_riopt.c	GAMS solver link (GMO API to ripoft C API)

The `augmented_lagrangian.rs` and `restoration.rs` modules that appeared in v0.7 have been folded into `ipm.rs` and `bc_solver.rs` for v0.8: the Gauss-Newton feasibility-solver loop, multi-attempt μ cascade, and post-restoration handoff now live in `ipm.rs` (search for `run_post_ls_restoration_cascade`, `apply_restoration_success`, `attempt_nlp_restoration`, `attempt_soft_restoration`), while the AL outer loop is driven from `bc_solver.rs`. The NE-to-LS reformulation (`detect_ne_problem`, `LeastSquaresProblem`) that v0.7 used was removed in T2.13 of the v0.8 Ipoft-alignment plan; the wins it previously claimed survived the removal because ripoft's implicit-slack IPM accepts the same problems as ordinary

equality-constrained NLPs.

2 Implicit Slack Formulation

2.1 Algorithm Description

The implicit slack formulation eliminates explicit slack variables for inequality constraints. Instead of introducing variables s and enforcing $g(x) - s = 0$, $g_L \leq s \leq g_U$, the algorithm treats the constraint slacks $s_L = g(x) - g_L$ and $s_U = g_U - g(x)$ as implicit functions of x .

For each inequality constraint i , the algorithm computes a barrier stiffness $\Sigma_{s,i}$ from the implicit slack multipliers, which are derived from the Lagrange multiplier y_i . When $y_i < 0$ (for a lower-bounded constraint), $z_{s_L,i} = -y_i$; otherwise, $z_{s_L,i} = \mu/\tilde{s}_{L,i}$ where $\tilde{s}_{L,i}$ is a safeguarded slack.

The barrier stiffness enters the (2,2) block of the KKT matrix as $-\Sigma_s^{-1}$, producing a system of dimension $(n + m)$ instead of $(n + m + n_{\text{ineq}})$.

When a constraint is infeasible ($s_{L,i} < 0$), no barrier term exists and the constraint receives a pure feasibility-driving right-hand side.

2.2 Source Code References

- KKT assembly function: `src/kkt.rs:160 (assemble_kkt)`
- Σ_s computation: `src/kkt.rs:323 (=let mut sigma_s = 0.0=)`
- (2,2) block entry written: `src/kkt.rs:388 (matrix.add(n + i, n + i, -sigma_s_inv))`
- Mathematical derivation comment: `src/kkt.rs:129`

2.3 Condensed KKT via Schur Complement (legacy)

In v0.7 and earlier, the IPM dispatched to a dense Schur-complement reduction of the KKT system when $m \geq 2n$ and $n \leq 100$. The reduced system $S\Delta x = r$ had dimension n instead of $n + m$, with $S = H + \Sigma_x + \delta_w I + J^T(-D_c)^{-1}J$, and gave 100–800× speedups on tall-narrow problems like EXPFITC ($n = 5$, $m = 502$).

The `CondensedKktSystem` struct is preserved in `src/kkt.rs` for reference, but no callsite invokes it in v0.8 — the dispatch was retired as part of the Ipopt-alignment effort because the kernel had no Ipopt counterpart and the tall-narrow speedups were not reproducible against the strict-Optimal convergence test. The dense vs. sparse decision in v0.8 is governed solely by `options.sparse_threshold` (default 110), applied to $n + m$.

- `CondensedKktSystem` struct (preserved, unused): `src/kkt.rs:2257`
- Active dense/sparse threshold ($n + m \geq \text{sparse_threshold}$): `src/ipm.rs:8145`

3 Convergence Test

3.1 Algorithm Description

riopt uses the same convergence test as Ipopt 3.14.x: two independent checks on the iterative multipliers, both of which must pass at the current iterate for `Optimal` status. Earlier ripoft releases computed auxiliary stationarity-based bound multipliers z_{opt} and used them in a separate scaled gate; v0.7.0 dropped that machinery after confirming that the iterative z_L, z_U (with fraction-to-boundary updates and Ipopt’s κ_σ safeguarding) are sufficient once the barrier subproblem has been driven to a small μ .

3.1.1 Dual Scaling s_d

Following Ipopt, the scaled tolerances for the dual-infeasibility and complementarity gates are divided by

$$s_d = \max\left(s_{\text{max}}, \frac{\|y\|_1 + \|z_L\|_1 + \|z_U\|_1}{n + m}\right) / s_{\text{max}},$$

with $s_{\text{max}} = 100$. The $s_d \leq 10^4$ cap that earlier ripoft releases used was removed in v0.7.1 to match Ipopt’s `IpIpoptCalculatedQuantities.cpp::scaled_dual_inf_x_L_violation` behavior; multipliers that grow large enough to mask a non-converged iterate are now handled by the unscaled gate (point 2 below) rather than by clipping the scaling.

3.1.2 Two-Gate Test

Convergence at the user tolerance ϵ requires **both**:

1. **Scaled gate:** $E_\mu / s_d \leq \epsilon$ for dual infeasibility and complementarity; primal infeasibility against ϵ directly.
2. **Unscaled gate:** primal infeasibility $\leq \text{constr_viol_tol}$, dual infeasibility $\leq \text{dual_inf_tol}$, complementarity $\leq \text{compl_inf_tol}$.

All three residuals use the iterative z values (matching Ipopt’s `curr_dual_infeasibility`). An **acceptable** level mirroring Ipopt’s defaults (`acceptable_tol` = 10^{-6} , 15 consecutive iterations, relaxed unscaled tolerances of 10^{-2}) is also tracked, but ripoft returns `Optimal` only when the strict two-gate test passes at the user tolerance.

3.2 Source Code References

- `check_convergence` function: `src/convergence.rs:117`
- `check_convergence_with_last_obj`: `src/convergence.rs:128`
- `ConvergenceInfo` struct: `src/convergence.rs:23`
- Dual scaling s_d : `src/convergence.rs:141`

- Complementarity scaling s_c : src/convergence.rs:146
- Acceptable-level helper (informational, **not** counted as solved): src/convergence.rs:72 (meets_acceptable_thresholds)

4 Two-Phase Restoration with Multi-Attempt Recovery

4.1 Algorithm Description

4.1.1 Phase 1: Gauss-Newton Feasibility Solver

The GN phase minimizes $\frac{1}{2}\|h(x)\|^2$ where $h_i(x) = \max(g_{L,i} - g_i, g_i - g_{U,i}, 0)$. Steps:

1. Restrict the Jacobian to violated constraints: J_a .
2. Solve $(J_a J_a^T + \epsilon I)w = h_a$ via Levenberg-Marquardt regularization.
3. Compute the step $\Delta x = -J_a^T w$.
4. Backtracking line search on $\|h\|^2$.

Fallback mechanisms:

- **Gradient descent**: When GN line search fails, use $\Delta x = -J^T h$ with separate backtracking.
- **Proximity regularization**: Add pull toward starting point when progress stalls.
- **Penalty-regularized gradient descent**: Minimize $\frac{1}{2}\|h\|^2 + \rho\|x - x_{\text{ref}}\|^2$.

4.1.2 Phase 2: NLP Restoration

If GN fails on two consecutive occasions, the full NLP restoration subproblem is solved:

$$\min_{x,p,n} \rho \sum (p_i + n_i) + \frac{\eta}{2} \|D_R(x - x_R)\|^2, \quad g(x) - p + n = g_{\text{target}}, \quad p, n \geq 0$$

The subproblem is solved by the same IPM engine with `=disable_nlp_restoration=true=` to prevent recursion. Dynamic dispatch (`&dyn NlpProblem`) breaks the monomorphization chain.

4.1.3 Multi-Attempt Recovery

When both phases fail, 6 recovery attempts are tried with alternating μ factors [10, 0.1, 100, 0.01, 1000, 0.001] and deterministic x perturbations starting at attempt 3. After all attempts, infeasibility is checked via $\|J^T h(x)\|_\infty < 10^{-4} \cdot \max(\theta, 1)$.

4.1.4 Post-Restoration State Reset

After successful restoration: $z_i = \mu/s_i$ (capped at 10^3), $y_i = 0$, barrier mode reset to Free, filter cleared.

4.2 Source Code References

The v0.7 `src/restoration.rs` module was folded into `src/ipm.rs` during the v0.8 alignment effort so that the GN feasibility loop, NLP restoration sub-NLP, and multi-attempt μ cascade live next to the IPM state they mutate.

- `RestorationNlp` struct (Phase 2 NLP): `src/restoration_nlp.rs:21`
- `RestorationNlp::new`: `src/restoration_nlp.rs:126`
- Phase 2 invocation: `src/ipm.rs:10381` (`attempt_nlp_restoration`)
- Soft-restoration sub-NLP entry: `src/ipm.rs:5628` (`attempt_soft_restoration`)
- Multi-attempt μ cascade driver: `src/ipm.rs:5288` (`run_post_ls_restoration_cascade`)
- μ factors cycle `[10, 0.1, 100, 0.01, 1000, 0.001]`: `src/ipm.rs:5187`
- Post-restoration state reset: `src/ipm.rs:10203` (`apply_restoration_success`)

5 Multi-Solver Fallback Architecture

5.1 Algorithm Description

When the primary IPM fails (`RestorationFailed`, `MaxIterations`, or `NumericalError`), up to three alternative solvers are attempted:

1. **L-BFGS**: For unconstrained problems, tried before IPM (no barrier needed). For bound-constrained, tried as fallback with projected gradient. Uses $k = 10$ curvature pairs, Wolfe line search.
2. **Augmented Lagrangian (AL)**: For constrained problems. PHR formulation with penalty parameter ρ increasing by $10\times$ when violation fails to decrease. Each AL subproblem is solved by the IPM engine.
3. **Explicit Slack Reformulation**: For inequality-constrained problems. Converts $g_L \leq g(x) \leq g_U$ to $g(x) - s = 0$, $g_L \leq s \leq g_U$, solved by IPM with `=enable_slack_fallback=false=`. Triggered on any non-Optimal result for strict objective comparison.

5.1.1 Best-Point Tracking

Throughout the solve, the iterate with lowest dual infeasibility is tracked. At max iterations, this point is restored and checked against acceptable convergence criteria with relaxed primal tolerance.

5.1.2 Dual Stagnation Detection

If no dual infeasibility improvement occurs for 500+ iterations, the best-du point is restored and checked for acceptable convergence.

5.2 Source Code References

- L-BFGS solver entry: `src/lbfgs.rs:24` (`pub fn solve`)
- L-BFGS memory size: `src/lbfgs.rs:13` (`=LBFGS_M = 10=`)
- Bound-constrained / augmented-Lagrangian driver: `src/bc_solver.rs:123` (`solve_bc`)
- `SlackFormulation` struct: `src/slack_formulation.rs:17`
- `SlackFormulation::new`: `src/slack_formulation.rs:29`
- `SlackNlpProblem` impl: `src/slack_formulation.rs:74`
- Outer fallback dispatch (L-BFGS / AL / slack): `src/ipm.rs:3589` (`solve`)
- Inner IPM loop: `src/ipm.rs:8075` (`solve_ipm`)

6 Adaptive Barrier Parameter

6.1 Algorithm Description

6.1.1 Barrier Oracle with Mehrotra Integration

The barrier parameter update combines a Loqo-style oracle with Mehrotra predictor-corrector feedback. In Free mode, the base update is $\mu = \bar{c}/\kappa$ where $\bar{c}$ is the average complementarity and $\kappa = 10$. When Mehrotra predictor-corrector is enabled (default), the centering parameter $\sigma = (\mu_{\text{aff}}/\mu)^3$ from the affine scaling probe is stored and used in the next mu update via a geometric blend: $\mu = \mu_{\sigma}^{1-\sigma} \cdot \mu_{\text{Loqo}}^{\sigma}$ where $\mu_{\sigma} = \sigma \cdot \mu_{\text{current}}$. This allows aggressive mu decreases when the affine step is near full-length ($\sigma \rightarrow 0$) while staying conservative otherwise.

6.1.2 Mehrotra Predictor-Corrector and Gondzio Corrections

Enabled by default (`=mehrotra_pc=true=`, `=gondzio_mcc_max=3=`). After factoring the KKT system, an affine predictor step ($\mu = 0$) probes the Newton direction. The centering parameter σ determines how much to reduce μ in the corrector RHS. Up to 3 Gondzio centrality corrections further improve centering by driving outlier complementarity pairs back toward the central path.

6.1.3 Free and Fixed Modes

- **Free mode:** μ updated via oracle (with Mehrotra blend) each iteration; filter reset each iteration.
- **Fixed mode:** μ decreases monotonically via $\mu_{\text{new}} = \min(\kappa_\mu \mu, \mu^{\theta_\mu})$ with $\kappa_\mu = 0.2$, $\theta_\mu = 1.5$. Subproblem convergence checked via $\epsilon_{\text{barrier}} \cdot \mu$.
- Mode switching based on sliding window of 4 KKT error values (`=refs_red_fact = 0.999=`).

6.1.4 Fraction-to-Boundary

Free mode uses $\tau = \max(\tau_{\text{min}}, 1 - e_{\text{NLP}})$ (more aggressive than Ipopt's $1 - \mu$).

6.1.5 No-Bounds Special Case

When no variable has finite bounds, $\mu = \mu_{\text{min}} = 10^{-11}$ immediately.

6.2 Source Code References

- `MuMode` enum: `src/ipm.rs:924`
- `MuState` struct: `src/ipm.rs:932`
- Inner IPM loop entry: `src/ipm.rs:8075` (`solve_ipm`)
- μ mode switch helper: `src/ipm.rs:6670` (`switch_mu_mode`)

7 Filter Line Search

7.1 Algorithm Description

The filter line search follows Fletcher & Leyffer (2002) as adapted by Wachter & Biegler (2006).

7.1.1 Switching Condition

The switching condition $\alpha(-\nabla\phi^T d)^{s_\phi} > \delta\theta^{s_\theta}$ includes the step size α as a multiplier. As α decreases during backtracking, the switching condition eventually fails, allowing h -type acceptance.

7.1.2 Second-Order Correction (SOC)

SOC is attempted on every backtracking step where θ increases (not just the first). Up to $k_{\text{soc}} = 4$ SOC iterations are performed with convergence check $\theta_{\text{soc}} \geq \kappa_{\text{soc}}\theta_{\text{prev}}$ ($\kappa_{\text{soc}} = 0.99$).

7.1.3 Objective Safeguard

Reject trial points where $\phi_{\text{trial}} > \phi_{\text{current}} + 5(1 + |\phi_{\text{current}}|)$.

7.2 Source Code References

- `Filter` struct: `src/filter.rs:14`
- `is_acceptable`: `src/filter.rs:147`
- `switching_condition`: `src/filter.rs:185`
- `armijo_condition`: `src/filter.rs:212`
- `check_acceptability`: `src/filter.rs:265`
- `fraction_to_boundary`: `src/filter.rs:559`

8 Linear Algebra

8.1 Algorithm Description

8.1.1 Sparse Solver Dispatch

The IPM automatically selects the linear solver based on problem size. For systems with $n + m < \text{sparse_threshold}$ (default 110), a dense Bunch-Kaufman LDL^T factorization is used. For larger systems, the default v0.8 sparse solver is `feral`,[?] a pure-Rust multifrontal LDL^T solver developed in a separate repository. The legacy `rmumps` backend is preserved behind the opt-in `rmumps` Cargo feature for A/B comparison.

8.1.2 `feral`: pure-Rust Multifrontal LDL^T (v0.8 default)

`feral` is a pure-Rust multifrontal solver developed in its own repository[?] with an independent benchmark corpus and reference-solver test harness. Moving the factorization out of the `riopt` tree resolves the v0.7 entanglement where linear-solver failures could only be reproduced through `riopt`'s CUTEst workload. The v0.8 build pulls `feral` as a regular Cargo dependency and remains entirely Rust — there is no FFI step. `feral` implements:

1. **AMD / nested-dissection fill-reducing orderings.**[?] [?]
2. **Supernodal symbolic factorization** and amalgamation.
3. **Dense Bunch-Kaufman pivoting**[?] on each fully-summed front, with SIMD-accelerated GEMM in the trailing update.
4. **MC64-style matching-based scaling** available as a pre-factor pass.
5. **Iterative refinement** applied through the wrapping driver.

The `FeralLdl` wrapper translates `riopt`'s upper-triangle COO triplet store (`KktMatrix::Sparse`) into `feral`'s expected format and caches the symbolic analysis across calls with the same sparsity pattern; repeated factorizations during inertia correction therefore only perform numeric refactorization.

8.1.3 `rmumps`: Legacy Multifrontal LDL^T (`feature"rmumps"=`)

For A/B testing against the v0.7 default, `riopt` preserves the `rmumps` multifrontal solver as a workspace member behind the opt-in `rmumps` Cargo feature. `rmumps` is also pure Rust (CeCILL-C license) and adds a contribution-block 2×2 pivot search for indefinite KKT systems plus a true incomplete- LDL^T preconditioner used by the legacy MINRES path (`src/linear_solver/iterative.rs`, `hybrid.rs`). Under v0.8's default `feral` backend the iterative driver currently degrades to direct factorization with extra iterative refinement; a true incomplete- LDL^T + MINRES path on top of `feral` is deferred to v0.9.

8.1.4 Bunch-Kaufman LDL^T Factorization (Dense)

For systems with dimension < 110 , a dense LDL^T factorization with Bunch-Kaufman pivoting is used. This produces $PLDL^TP^T = A$ where D has 1×1 and 2×2 blocks. The 2×2 blocks handle the indefinite structure of the KKT matrix.

A critical implementation detail: when rows/columns are swapped during pivoting, L entries from previously computed columns must also be swapped.

8.1.5 Inertia Correction

The KKT matrix must have inertia $(n, m, 0)$. After factorization, inertia is extracted from D and regularization δ_w (to $(1, 1)$ block) and $-\delta_c$ (to $(2, 2)$ block) is added if incorrect. The regularization grows exponentially ($\delta_w \rightarrow 4\delta_w$) for up to 15 attempts. On failure after all attempts, the algorithm proceeds with the approximate factorization.

8.1.6 Iterative Refinement

Up to 3 rounds of iterative refinement with tolerance 10^{-12} .

8.2 Source Code References

8.2.1 `feral` Integration (v0.8 default)

- `FeralLdl` struct: `src/linear_solver/feral_direct.rs:44`
- `LinearSolver` impl for `FeralLdl`: `src/linear_solver/feral_direct.rs:219`
- Iterative refinement driver around `feral`: `src/linear_solver/feral_iterative.rs`
- Hybrid direct/iterative driver around `feral`: `src/linear_solver/feral_hybrid.rs`

8.2.2 `rmumps` Integration (`legacy, feature"rmumps"=`)

- `MultifrontalLdl` struct: `src/linear_solver/multifrontal.rs:9`
- `LinearSolver` impl for `MultifrontalLdl`: `src/linear_solver/multifrontal.rs:116`
- Legacy iterative driver: `src/linear_solver/iterative.rs`
- Legacy hybrid driver: `src/linear_solver/hybrid.rs`

8.2.3 Trait surface

- `LinearSolver` trait: `src/linear_solver/mod.rs:622`
- `KktMatrix` enum (`Dense/Sparse`): `src/linear_solver/mod.rs:495`

8.2.4 Dense Solver

- `DenseLdl` struct: `src/linear_solver/dense.rs:11`
- `bunch_kaufman_factor`: `src/linear_solver/dense.rs:65`
- `find_pivot`: `src/linear_solver/dense.rs:225`
- `compute_inertia`: `src/linear_solver/dense.rs:299`
- `solve_internal`: `src/linear_solver/dense.rs:351`

8.2.5 Inertia Correction and Search Direction

- `InertiaCorrectionParams` struct: `src/kkt.rs:605`
- `factor_with_inertia_correction`: `src/kkt.rs:1067`
- `factor_with_inertia_correction_cached` (warm-restarted ladder): `src/kkt.rs:1240`
- `solve_for_direction` (basic): `src/kkt.rs:1393`
- `solve_for_direction_with_ir_ctx` (iterative refinement): `src/kkt.rs:1430`
- `solve_for_direction_iter_aware_with_ir_ctx` (iter-aware refinement): `src/kkt.rs:1993`

9 Implementation

9.1 Problem Interface

The `NlpProblem` trait defines a buffer-filling interface for optimization problems. All evaluation methods take pre-allocated slices to avoid heap allocation in the hot loop.

- `NlpProblem` trait: `src/problem.rs:17`

- `SolverOptions` struct: `src/options.rs`
- `SolveStatus` enum (`Optimal`, `Acceptable`, `MaxIterations`, `MaxTimeExceeded`, `RestorationFailed`): `src/result.rs`

9.2 Public API

The public API consists of two functions:

- `solve<P: NlpProblem>(problem: &P, options: &SolverOptions) -> SolveResult` for standard optimization.
- `solve_with_sensitivity<P: ParametricNlpProblem>(problem: &P, options: &SolverOptions) -> (SolveResult, SensitivityContext)` for optimization with parametric sensitivity analysis.
- `solve` function: `src/lib.rs:126`
- `solve_with_sensitivity` function: `src/lib.rs:155`
- Internal IPM dispatch (preprocessing, fallback selection, then `solve_ipm`): `src/ipm.rs:3589 (solve)`
- `solve_ipm` (core IPM loop): `src/ipm.rs:8075`

9.3 AMPL NL Interface and Pyomo Integration

The AMPL interface reads NL files (nonlinear expression DAGs) with automatic differentiation for gradient, Jacobian, and Hessian evaluation, and writes SOL files.

- NL file parser: `src/nl/parser.rs:139 (parse_nl_file)`
- `NlFileData` struct: `src/nl/parser.rs:45`
- `NlProblem` struct: `src/nl/problem_impl.rs:11`
- `NlpProblem` for `NlProblem` impl: `src/nl/problem_impl.rs:227`
- AMPL binary (reads `.nl`, writes `.sol`): `src/bin/riopt_ampl.rs`
- Pyomo plugin: `pyomo-riopt/` directory

9.4 Julia/JuMP Interface (Ripopt.jl)

`Ripopt.jl` wraps `ripost`'s C API behind a `MathOptInterface` (MOI) `Optimizer` type, enabling use from JuMP. The wrapper implements the incremental MOI interface so that `MOI.Utilities.default_copy_to` can populate the optimizer from JuMP's internal cache.

Key source files:

- Module entry point and `__init__` (runtime library path): `Ripopt.jl/src/Ripopt.jl`
- C API bindings (`RipoptProblem`, `CreateRipoptProblem`, `RipoptSolve`, options, statistics): `Ripopt.jl/src/C_wrapper.jl`
- MOI wrapper (`Optimizer` struct, `copy_to`, `optimize!`, result accessors): `Ripopt.jl/src/MOI_wrapper.jl`

Design notes:

- The five C callbacks (`eval_f`, `eval_grad_f`, `eval_g`, `eval_jac_g`, `eval_h`) are module-level functions, not closures. This is required on ARM/AArch64 (Apple Silicon) where Julia does not support trampoline-backed `@cfunction` closures (the `$f` form). All problem data is passed through the `user_data` pointer as a GC-protected `Ref{NamedTuple}`.
- The library path is a mutable global set in `__init__`, not a `const`. This ensures the `RIPOPT_LIBRARY_PATH` environment variable is read at module load time, so the precompiled module image is not tied to a specific build directory.
- `MOI.supports_incremental_interface` returns `true`; `copy_to` delegates to `MOI.Utilities.default_copy_to`. Variable bounds go through `MOI.VariablesContainer`; the nonlinear objective and constraints go through `MOI.NLPBlock`.
- Option dispatch checks `Integer` before `Real` to avoid routing integer-valued options (e.g., `max_iter`) to the floating-point option setter.

10 Parametric Sensitivity Analysis

10.1 Algorithm Description

The sensitivity analysis implements the sIPOPT approach: after the IPM converges, the KKT system is reassembled *without* inertia correction ($\delta_w = 0$, $\delta_c = 0$) and factored. For each parameter perturbation Δp , the RHS is assembled from user-provided parameter derivatives ($\nabla_{x_p}^2 L$ and $\partial g / \partial p$), and the sensitivity is obtained by a single backsolve.

The key design decision is to *not* retain the KKT matrix from the final IPM iteration. The final KKT may include inertia correction terms that would introduce $O(\delta)$ bias. Instead, we pay the cost of one extra factorization for exact sensitivities.

Bound multiplier sensitivities are derived from complementarity. From $z_L(x - x_L) = \mu$ at the barrier solution, differentiating with respect to p gives $dz_L = -z_L dx / (x - x_L)$. Similarly for upper bounds: $dz_U = z_U dx / (x_U - x)$.

The reduced Hessian is extracted column-by-column from M^{-1} by solving $Me_j = \text{col}_j$ for $j = 1, \dots, n$. The $n \times n$ upper-left block gives the projected Hessian inverse, useful for covariance estimation in parameter estimation problems.

10.2 Source Code References

- `ParametricNlpProblem` trait: `src/sensitivity.rs:19`
- `SensitivityContext` struct: `src/sensitivity.rs:39`
- `SensitivityResult` struct: `src/sensitivity.rs:56`
- `solve_with_sensitivity()` function: `src/sensitivity.rs:72`
- `compute_sensitivity()` backsolve: `src/sensitivity.rs:161`
- `reduced_hessian()` extraction: `src/sensitivity.rs:286`
- Integration tests (HS071 parametric, finite difference verification): `tests/sensitivity.rs`
- Example (HS071 with parametric constraint bound): `examples/sensitivity.rs`

11 Running the Benchmarks

11.1 Prerequisites

- Rust toolchain (stable): <https://rustup.rs>
- For CUTEst benchmarks: CUTEst library and Ipopt (with MUMPS) installed
- For HS comparisons with cyipopt: Python 3 with `cyipopt` package

11.2 Hock-Schittkowski (HS) Suite

11.2.1 Generating HS Problem Code

The HS problems are generated from Fortran source files:

```
1 cd benchmarks/hs
2 python generate.py # generates benchmarks/hs/generated/hs_problems.rs
```

11.2.2 Running ripopt on HS Suite

```
1 cargo run --release --features hs --bin hs_suite
```

This solves the HS problems and outputs JSON results to stdout. The number of timing runs can be controlled via environment variable:

```
1 RIPOPT_TIMING_RUNS=10 cargo run --release --features hs --bin hs_suite \  
2 > benchmarks/hs/hs_riopt_results.json
```

To force sparse factorization on all problems (for comparison):

```
1 RIPOPT_FORCE_SPARSE=1 cargo run --release --features hs --bin hs_suite
```

11.2.3 Running Ipoct (native) on HS Suite

Requires the `ipoct-native` feature and Ipoct C library:

```
1 cargo run --release --features ipoct-native --bin ipoct_native \  
2 > benchmarks/hs/hs_ipoct_native_results.json
```

11.2.4 Running cyipoct on HS Suite

```
1 cd benchmarks/hs  
2 python run_cyipoct.py > hs_cyipoct_results.json
```

11.2.5 Comparing Results

```
1 cd benchmarks/hs  
2 python compare.py
```

This reads the JSON results and generates `HS_VALIDATION_REPORT.md` with solve rates, objective comparisons, speed comparisons, and iteration counts. A second, more detailed report is written to `benchmarks/hs/PERFORMANCE_COMPARISON.md`.

11.3 CUTEst Benchmark Suite

11.3.1 Prerequisites

CUTEst must be installed and the environment configured so that the CUTEst shared libraries and SIF files are available. The `CUTEST` and `MASTSIF` environment variables should point to the CUTEst installation and SIF problem collection respectively.

11.3.2 Running CUTEst Benchmarks

To run specific problems:

```
1 cargo run --release --features cutest,ipoct-native --bin cutest_suite -- ROSENBR HS35 HS71
```

To run all problems from the problem list:

```
1 RESULTS_FILE=benchmarks/cutest/results.json \  
2 cargo run --release --features cutest,ipoct-native --bin cutest_suite
```

This reads problem names from `benchmarks/cutest/problem_list.txt` and solves each with both ripopt and native Ipopt. Results are auto-saved to `benchmarks/cutest/results.json` (override with `RESULTS_FILE`).

The solver uses `=max_wall_time=30s=` so the IPM exits before the 60-second OS kill limit, enabling fallback solvers to attempt recovery.

11.3.3 Interpreting Results

Results are saved in JSON format with fields:

- `name`: CUTEst problem name
- `solver`: "riopt" or "ipopt"
- `n, m`: problem dimensions
- `status`: solver status string
- `objective`: final objective value
- `constraint_violation`: final constraint violation
- `iterations`: number of iterations
- `solve_time`: wall-clock time in seconds

11.4 Unit Tests

The correctness test suite covers fundamental problem types:

```
1 cargo test --release
```

Test problems include Rosenbrock, simple QP, HS071, bound-constrained QP, multi-equality, and pure bound problems. See `tests/correctness.rs` for details.

11.5 Pyomo Integration

To test the Pyomo integration:

```
1 # Build the ripopt AMPL binary
2 cargo build --release --bin ripopt
3
4 # Install the Pyomo plugin
5 cd pyomo-riopt
6 pip install -e .
7
8 # Test
9 python -c "
10 import pyomo_riopt
11 from pyomo.environ import *
12 model = ConcreteModel()
13 model.x = Var(initialize=1.5, bounds=(1, 5))
```

```

14 model.y = Var(initialize=1.5, bounds=(1, 5))
15 model.obj = Objective(expr=(model.x - 1)**2 + (model.y - 2)**2)
16 model.c1 = Constraint(expr=model.x + model.y <= 4)
17 solver = SolverFactory('ripopt')
18 result = solver.solve(model, tee=True)
19 print(f'x={value(model.x):.6f}, y={value(model.y):.6f}, obj={value(model.obj):.6f}')
20 "

```

11.6 Julia/JuMP Integration

To run the Julia examples (requires Julia 1.9 with JuMP installed):

```

1 # Build the shared library
2 cargo build --release
3
4 # Run the JuMP examples
5 RIPOPT_LIBRARY_PATH=target/release julia --project=@v1.12 \
6   Ripopt.jl/examples/jump_hs071.jl
7
8 RIPOPT_LIBRARY_PATH=target/release julia --project=@v1.12 \
9   Ripopt.jl/examples/jump_rosenbrock.jl
10
11 # Low-level C wrapper example (no JuMP required)
12 RIPOPT_LIBRARY_PATH=target/release julia --project=@v1.12 \
13   Ripopt.jl/examples/c_wrapper_hs071.jl

```

Expected output for `jump_hs071.jl`:

```

Termination status: LOCALLY_SOLVED
Objective value:    17.014017289919348
Solution:          [1.0, 4.743, 3.821, 1.379]

```

To install `Ripopt.jl` into a Julia environment:

```

1 import Pkg
2 Pkg.develop(path="Ripopt.jl") # or Pkg.add(url="...") for remote

```

The interactive notebook tutorial is at `Ripopt.jl/examples/ripopt_jump_tutorial.ipynb`.